SC-MODEO-CNN: Imbalance-Aware, Zero-Day-Capable and Explainable Neural Architecture Search for IIoT Intrusion Detection

Yaswitha Nandu Yalamanchali, T. Harshini Sri Savitha, Pradeep Kumar Gajendra Bhale
IIITDM Kancheepuram

Abstract

Attacks on Industrial Internet of Things (IIoT) and industrial control systems can change physical processes, so intrusion detectors for these networks must be both accurate and trustworthy. MODEO-CNN [1] designs lightweight convolutional detectors automatically through multi-objective neural architecture search, but it ignores class imbalance, assumes every attack seen at test time was seen in training, and gives no reason for its alerts. We propose SC-MODEO-CNN, which adds a loss-function gene and a macro-F1 objective for imbalance, a third objective that rewards separating a held-out pseudo-unknown attack, a fused zero-day detector with per-class thresholds, and attribution-based explanations checked for faithfulness and stability. On the Gas Pipeline dataset, with DoS withheld as an unseen attack, five seeds and an equal search budget, SC-MODEO-CNN reaches a macro-F1 of 0.977 against 0.974. The fraction of DoS traffic accepted as known falls from 0.21–0.42 to about 0.04, open-set macro-F1 rises from 0.66–0.69 to 0.88–0.89, and 97% of DoS records are flagged. Deleting the highest-attributed features degrades predictions faster than random deletion in all ten models. The closed-set gain is small and not significant with five seeds. **Keywords—** Intrusion detection, Industrial Internet of Things, neural architecture search, multi-objective optimization, open-set recognition, explainable AI.

1 Introduction

Industrial Internet of Things (IIoT) and industrial control systems (ICS) run gas pipelines, water treatment plants and power grids. Many of them still communicate over legacy protocols such as Modbus, which were built for closed networks and offer little or no authentication. An attacker who reaches the network can therefore inject false process values or commands, and the consequence is not only lost data but possible physical damage to equipment and people. Deep-learning intrusion detection systems (IDS) have shown strong accuracy on such traffic, yet most are designed by hand, which demands expert effort and often produces models that are too heavy for resource-limited industrial edge devices.

One way to remove the manual step is to search for the architecture and its hyperparameters jointly, combining neural architecture search (NAS) with hyperparameter optimisation (HPO). Huang et al. first did this for IIoT with SOPA-GA-CNN [2], a genetic algorithm over CNN blocks. Its successor, MODEO-CNN [1], encodes a candidate network as a variable-length sequence of blocks and optimises it with multi-objective discrete extremal optimisation, minimising classification error and computational cost (MFLOPs) together. On the Gas Pipeline, BoT-IoT and Power System datasets it reports models that are both lightweight and accurate.

Three gaps remain. First, IIoT traffic is highly imbalanced: in Gas Pipeline, the MFCI and MSCI attacks make up only 0.59% and 0.81% of the records. MODEO-CNN selects models by weighted F1, which is dominated by the large classes, so a model can fail on a rare attack and still score well. Second, MODEO-CNN is a closed-set classifier. A deployed IDS will meet attacks absent from its training data, but a closed-set model must assign every input to one of the known classes [3], so a new attack is silently labelled as something familiar. Third, the detector is a black box: it raises an alert without saying why, which makes it hard for operators to trust the decision or act on it.

Reimplementing MODEO-CNN also exposed two practical problems. No public code is available, so we rebuilt the method from the paper. On Gas Pipeline, some feature values reach $\pm 2.3 \times 10^{38}$, and the L2 normalisation used in the paper collapses every other feature to almost zero (problem A). In addition, the search stalled on tiny models that predict only the majority class (problem B). Section 4 describes how we fixed both.

The main contributions of this work are as follows:

- A working reimplementation of MODEO-CNN, made usable on Gas Pipeline by a signed-logarithm preprocessing step and a feasibility rule that removes degenerate candidates.
- SC-MODEO-CNN, which handles imbalance through a loss-function gene and a macro-F1 objective, and adds a third objective f_3 that rewards zero-day separability, using a pseudo-unknown class chosen automatically without looking at test data.
- A fused zero-day detector that combines a Mahalanobis score on the CNN embedding with autoencoder reconstruction error and uses per-class thresholds, missing about ten times fewer DoS records.
- Explanations whose faithfulness (deletion/insertion) and stability (Kendall τ) are measured, and a full evaluation

with five seeds, four ablations, hand-designed baselines and Wilcoxon tests.

The rest of the paper is organised as follows. Section 2 reviews related work and Section 3 summarises MODEO-CNN. Section 4 presents SC-MODEO-CNN, and Section 5 describes the experimental setup. Section 6 reports the results, Section 7 discusses them together with the limitations, and Section 8 concludes.

2 Related Work

2.1 NAS for intrusion detection

Evolutionary NAS with variable-length CNN encodings was popularised in image classification by Sun et al. [4], where the depth of the network evolves along with its layers. For IIoT intrusion detection, SOPA-GA-CNN [2] applied a genetic algorithm to optimise architecture and parameters together, but under a single objective. MODEO-CNN [1] replaced the genetic algorithm with extremal optimisation [5] and kept an external Pareto archive pruned by crowding distance [6], trading accuracy against model size. None of these methods targets class imbalance, unseen attacks or explanations, which are the focus of this work.

2.2 Zero-day and open-set detection

Open-set recognition asks a classifier to label known classes correctly while rejecting inputs from classes it has never seen [3]. A common practical approach scores each input after training, without changing the classifier: the maximum softmax probability (MSP) [7], the energy of the logits [8] and the Mahalanobis distance of intermediate features to class means [9] are standard choices. In network intrusion detection, reconstruction error from autoencoders trained on benign or known traffic is another established signal, as in Kitsune [10]. We combine the two families, a feature-space distance from the searched CNN and a reconstruction error from a separate autoencoder.

2.3 Class imbalance and explainability

Focal loss [11] down-weights easy, well-classified samples so training concentrates on hard ones, and class-balanced loss [12] re-weights classes by their effective number of samples. For explanation, Integrated Gradients (IG) [13] and SHAP [14] attribute a prediction to individual input features. Whether such attributions reflect what the model actually uses can be tested with deletion and insertion curves [15]. We compute all attributions with Captum [16].

3 Background: MODEO-CNN

MODEO-CNN represents a candidate network as $[n, topo, conv_1, \ldots, conv_n, setting]$. The number of blocks n lies in $[C_L, C_H] = [3, 10]$. The topology part $topo$ holds $n(n-1)/2$ bits, one for each possible link from an earlier block to a later one. Each block is described by five integer genes: number of filters, kernel size, activation, pooling type and pool size. The setting part fixes the batch size, optimiser, learning rate and regulariser. The integer genes are decoded into actual values with Eq. (1).

$$\begin{aligned} &\text{channels} = 16\,(k_{\text{num}} + 1), \quad \text{kernel} = 2\,k_{\text{size}} + 3, \\ &\text{pool size} = 2\,p_s + 3, \quad \text{batch} = b\,(b_s + 1), \\ &\eta = 2.5 \times 10^{-4}\,(lr + 1). \end{aligned} \tag{1}$$

The search keeps a single current solution S_N. In each generation it creates $n + 3$ mutants of S_N: one that re-initialises the network, one with a new topology, one per block in which only that block is redrawn, and one with a new setting. Every mutant is trained and evaluated, the first Pareto front among the mutants is found, and a random member of this front becomes the next S_N. Non-dominated solutions are collected in an external archive of at most $A_{\max}$ members, pruned by crowding distance when it overflows. The original objectives are $1 - F_1$ and MFLOPs; the original paper does not state how F_1 is averaged, and we use weighted F_1.

4 Proposed Method: SC-MODEO-CNN

SC-MODEO-CNN makes six changes to MODEO-CNN, summarised in Algorithm 1. The first four (preprocessing, the loss gene with a macro-F1 objective, the feasibility rule and the zero-day objective f_3) act inside the search. The last two (the fused zero-day detector and explainability) are applied to the trained models and require no retraining.

4.1 Preprocessing for heavy-tailed features

The `measurement` field of Gas Pipeline ranges from about -1.5×10^{38} to $+2.3 \times 10^{38}$. Under the row-wise L2 normalisation used in [1], such a value dominates the norm and every other feature in the row shrinks to nearly zero. The effect is severe: only 4% of random genomes learned anything useful (median macro-F1 0.10), and even a fixed, hand-designed CNN reached only 0.47. We therefore apply the signed logarithm of Eq. (2), which keeps the order and sign of values but compresses their range, followed by z-score standardisation. With this change, 21% of random genomes learn and the fixed CNN reaches 0.95. We also drop eight constant columns and keep the time field, which in this release is already the interval between packets, which leaves 18 features.

$$x' = \operatorname{sign}(x)\,\log\bigl(1 + |x|\bigr). \tag{2}$$

4.2 Imbalance-aware objective and loss gene

We add a fifth gene to the setting part that selects the training loss: cross-entropy, weighted cross-entropy, class-balanced

Algorithm 1 SC-MODEO-CNN search (changes to MODEO-CNN marked ▷)

Require: training data D with test-unknown class U removed; G, $A_{\max}$, $[C_L, C_H]$, margin δ

$P \leftarrow \arg\min_{c \neq U} \mathrm{AUROC}_{\mathrm{raw}}(c)$ ▷▷ pseudo-unknown, Eq. (5)
split D into D_{tr} (without P) and D_{va} (known part D_{va}^{k}, P part D_{va}^{P})
$S_N \leftarrow$ random candidate (setting includes loss gene) ▷▷
$A \leftarrow \emptyset$
for $t = 1$ **to** G **do**
  $\mathcal{P}_t \leftarrow$ the $n+3$ mutants of S_N
  for all $m \in \mathcal{P}_t$ **do**
    train m on D_{tr} with the loss selected by its gene ▷ ▷
    $f_1 \leftarrow 1 -$ macro-$F_1(m, D_{va}^{k})$; $f_2 \leftarrow \mathrm{MFLOPs}(m)$ ▷▷
    $f_3 \leftarrow 1 - \mathrm{AUROC}\big(s_{\mathrm{Maha}}(D_{va}^{k}), s_{\mathrm{Maha}}(D_{va}^{P})\big)$ ▷ ▷
    if m is infeasible (Eq. (4)) **then** $f_2 \leftarrow 10^9$; $f_3 \leftarrow 1$ ▷▷
    end if
  end for
  $S_N \leftarrow$ random member of the first Pareto front of $\mathcal{P}_t$
  $A \leftarrow$ non-dominated set of $A \cup \{S_N\}$, pruned to $A_{\max}$ by crowding distance
end for
return A; fully train selected members on all known classes

cross-entropy ($\beta = 0.999$) or focal loss ($\gamma \in \{1, 2\}$), as defined in Eq. (3). Here n_c is the number of samples in class c, K the number of classes and p_t the predicted probability of the true class. The mutation operators are unchanged; the setting mutant simply resamples this gene along with the others. The first objective becomes $f_1 = 1 -$ macro-F_1, so that every class counts equally, regardless of its size.

$$\begin{aligned} \mathcal{L}_{\mathrm{wCE}} &= -w_y \log p_t, & w_c &= N/(K\, n_c), \\ \mathcal{L}_{\mathrm{CB}} &= -w_y \log p_t, & w_c &\propto (1-\beta)/(1-\beta^{n_c}), \\ \mathcal{L}_{\mathrm{focal}} &= -(1-p_t)^{\gamma} \log p_t. \end{aligned} \tag{3}$$

4.3 Feasibility rule

In early runs we observed that a model predicting only the majority class has the fewest MFLOPs and therefore always stays on the Pareto front. Because the search follows a single current solution, it repeatedly picked such models and got stuck: in the first runs every candidate had $1 - F_1 = 0.864$. We therefore call a model feasible only if it beats the majority predictor by a margin $\delta = 0.05$, as stated in Eq. (4). An infeasible model receives the worst possible values of f_2 and f_3, so any useful model dominates it. The rule is applied to both MODEO-CNN and SC-MODEO-CNN, so the comparison remains fair.

$$m \text{ feasible} \iff F_1(m) \geq F_1(\text{majority predictor}) + \delta. \tag{4}$$

4.4 Zero-day objective and pseudo-unknown selection

To make the search value zero-day detection, we hide one attack class P from the training data used during search and score it only through f_3, which measures how well a Mahalanobis score on the CNN embedding separates P from the known classes. Because P differs from the test-time unknown U, no information about U leaks into the search. The choice of P matters. Recon, for example, is already separable on the raw features (AUROC 1.000), so $f_3 = 0$ for every model and the objective gives no signal. We therefore pick the hardest class on the training data, following Eq. (5). For $U =$ DoS, this selects CMRI (AUROC 0.900); the next hardest is NMRI (0.942), and the remaining classes score between 0.97 and 1.00.

$$\begin{aligned} P &= \arg\min_{c \notin \{\text{Normal},\, U\}} a_c, \\ a_c &= \mathrm{AUROC}\big(s^{\mathrm{raw}}_{\mathrm{Maha}}(X_{\neg c}),\, s^{\mathrm{raw}}_{\mathrm{Maha}}(X_c)\big), \end{aligned} \tag{5}$$

where X_c are the training records of class c and $X_{\neg c}$ the remaining known records.

4.5 Fused zero-day detector with per-class thresholds

All scores in Eq. (6) follow one convention: a higher value means the input is more likely unknown. MSP and energy are computed from the logits ℓ. The Mahalanobis score uses class means μ_c and a shared covariance Σ estimated on the CNN embedding z. The autoencoder score is the reconstruction error of an encoder h and decoder g trained on the known classes only. To fuse the Mahalanobis and autoencoder scores, we take the logarithm of each, z-normalise it with statistics from the calibration set, and average the two.

$$\begin{aligned} s_{\mathrm{MSP}} &= -\max_k \mathrm{softmax}(\ell)_k, \\ s_E &= -\log \textstyle\sum_k e^{\ell_k}, \\ s_{\mathrm{Maha}} &= \min_c (z-\mu_c)^{\top} \Sigma^{-1} (z-\mu_c), \\ s_{\mathrm{AE}} &= \tfrac{1}{d} \lVert x - g(h(x)) \rVert^2, \\ s_{\mathrm{fus}} &= \tfrac{1}{2} \textstyle\sum_{i \in \{\mathrm{Maha},\mathrm{AE}\}} \big(\log s_i - m_i\big)/\sigma_i. \end{aligned} \tag{6}$$

A single global threshold at the 95th percentile of calibration scores works poorly for rare known classes such as MSCI and MFCI, whose samples naturally score high and are therefore rejected as unknown. We instead set one threshold per predicted class, as in Eq. (7), and fall back to the global threshold when a class has fewer than 20 calibration samples. Only calibration data from known classes is used to set the thresholds.

$$\begin{aligned} \tau_c &= Q_{0.95}\{\, s(x) : x \in D_{\mathrm{cal}},\ \hat{y}(x) = c \,\}, \\ &\text{reject } x \text{ if } s(x) > \tau_{\hat{y}(x)}. \end{aligned} \tag{7}$$

4.6 Explainability

We explain each prediction with Integrated Gradients (Eq. (8)), using the training mean as baseline x' and 32 in-

Table 1: Gas Pipeline dataset [17]: class distribution.

Class	Description	Samples	Share (%)
Normal	Normal traffic	61 156	63.04
NMRI	Naive malicious response injection	2 763	2.85
CMRI	Complex malicious response injection	15 466	15.94
MSCI	Malicious state command injection	782	0.81
MPCI	Malicious parameter command injection	7 637	7.87
MFCI	Malicious function code injection	573	0.59
DoS	Denial of service	1 837	1.89
Recon	Reconnaissance	6 805	7.01
Total		97 019	100

Table 2: Search settings: original paper vs. this study (identical for every method we compare).

Setting	MODEO-CNN paper	This study
Generations G	200	40
Epochs per fitness evaluation	30	5
Training data used during search	100%	30% (stratified)
Final training epochs	350	60
Archive size $A_{\max}$	20	20
Blocks $[C_L, C_H]$	$[3, 10]$	$[3, 10]$
Base batch size b	256	256
Independent runs (seeds)	–	5
Hardware	–	NVIDIA T4 (Kaggle)

tegration steps, and with GradientSHAP [14]; both explain the predicted class. From these we report global and per-class feature importance. Faithfulness is checked with deletion and insertion curves, compared against a random feature order [15]. Stability is measured by Kendall τ between feature rankings of different models. For an input flagged as unknown, we report the nearest known class together with its most important features, giving the operator a starting point for investigation.

$$\mathrm{IG}_j(x) = (x_j - x'_j) \int_0^1 \frac{\partial F\big(x' + \alpha(x - x')\big)}{\partial x_j}\, d\alpha. \qquad (8)$$

5 Experimental Setup

5.1 Dataset and protocol

We use the Gas Pipeline dataset [17] (Table 1), which contains 97,019 Modbus records in eight classes and is strongly imbalanced, with MFCI at 0.59% and MSCI at 0.81%. The data is split into 80% training and 20% test, stratified by class. DoS is removed from the training data, so its test rows form the unseen attack set. A further 10% of the training data is held out as a calibration set for setting thresholds. During the search, candidates are trained on a stratified 30% subsample of the training data, itself split 80/20 into training and validation. We hold out DoS because it is behaviourally distinct from the injection attacks; a leave-one-attack-out (LOAO) protocol over all attack classes is left for future work and is in progress.

5.2 Compared methods and budget

We compare our reimplementation of MODEO-CNN (weighted F1, cross-entropy loss) with SC-MODEO-CNN.

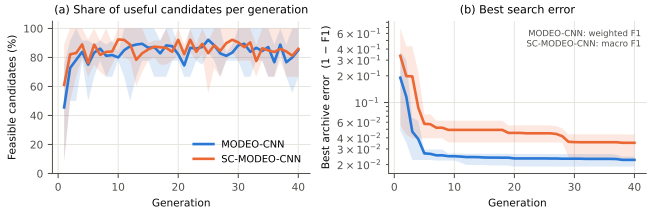


Figure 1: Search dynamics over 40 generations (mean and min–max over 5 seeds). Panel (b) uses weighted F1 for MODEO-CNN and macro F1 for SC-MODEO-CNN.

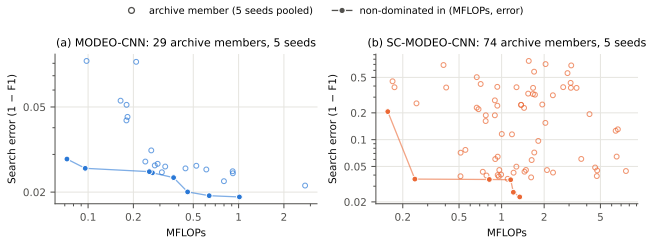


Figure 2: Final archives (5 seeds pooled) projected on MFLOPs and search error. SC-MODEO-CNN archives are three-objective fronts, shown here in projection.

Four ablations remove or alter one component each: no loss gene, no f_3, f_3 computed with MSP instead of Mahalanobis, and an easy pseudo-unknown (Recon). As hand-designed baselines we use a CNN with three layers of 64 filters trained with Adam, an LSTM and an autoencoder. Because of limited compute, the search budget is smaller than in [1] but identical for every method (Table 2). Each method is run with five seeds on an NVIDIA T4 GPU on Kaggle, and one search takes 9–12 minutes. Some details are not specified in [1]; we assume 1-D convolutions, concatenation of block inputs, stride-2 pooling and an L1/L2 regularisation strength of 10^{-4}.

5.3 Metrics

For closed-set classification on the seven known classes we report accuracy, weighted F1 and macro F1. For zero-day detection we report AUROC with the unknown class as positive; FPR@95, the fraction of DoS records accepted as known when 95% of known traffic is accepted; the DoS detection rate; and open-set macro-F1 over the seven known classes plus "unknown". Model size is measured in MFLOPs. Methods are compared with the paired Wilcoxon signed-rank test [18]; with five paired runs, the smallest attainable two-sided p-value is 0.0625. For each run we report the archive member with the best performance.

6 Results

6.1 Search behaviour

Figure 1(a) shows that, with the feasibility rule, the search quickly concentrates on useful models: the share of feasible

Table 3: Closed-set classification on the seven known Gas Pipeline classes (DoS held out), best-performance solution, mean ± s.d. over 5 seeds.

Model	Macro-F1	Weighted-F1	Accuracy	MFLOPs	Search (min)
MODEO-CNN (reimpl.)	0.974 ± 0.005	0.983 ± 0.006	0.983 ± 0.006	1.066 ± 1.005	9.134 ± 5.190
SC-MODEO-CNN (ours)	0.977 ± 0.004	0.987 ± 0.005	0.987 ± 0.005	1.335 ± 0.211	11.792 ± 3.455
Hand-designed CNN	0.980 ± 0.002	0.991 ± 0.000	–	–	–
LSTM	0.978 ± 0.002	0.989 ± 0.001	–	–	–

Table 4: Zero-day detection of the unseen DoS attack. Thresholds accept 95% of known calibration traffic (globally, or per predicted class). Mean ± s.d. over 5 seeds.

Model	Score	AUROC ↑	FPR@95 ↓	DoS detected ↑	Open-set macro-F1 ↑
MODEO-CNN (reimpl.)	Mahalanobis	0.928 ± 0.052	0.212 ± 0.142	0.790 ± 0.142	0.715 ± 0.032
	Autoencoder	0.985 ± 0.010	0.025 ± 0.018	0.972 ± 0.024	0.649 ± 0.029
	Fusion	0.984 ± 0.009	0.038 ± 0.028	0.963 ± 0.027	0.687 ± 0.030
	Fusion + per-class thr.	0.984 ± 0.009	0.038 ± 0.028	0.968 ± 0.023	0.885 ± 0.006
SC-MODEO-CNN (ours)	Mahalanobis	0.897 ± 0.084	0.419 ± 0.315	0.580 ± 0.319	0.709 ± 0.041
	Autoencoder	0.985 ± 0.010	0.025 ± 0.018	0.972 ± 0.024	0.651 ± 0.029
	Fusion	0.983 ± 0.012	0.040 ± 0.039	0.959 ± 0.044	0.664 ± 0.051
	Fusion + per-class thr.	0.983 ± 0.012	0.040 ± 0.039	0.972 ± 0.025	0.881 ± 0.007

Table 5: Ablation of SC-MODEO-CNN components (DoS held out, 5 seeds).

Variant	Macro-F1	MFLOPs	AUROC (Mahalanobis)	Open-set F1 (fusion, per-class)
SC-MODEO-CNN (ours)	0.977 ± 0.004	1.335 ± 0.211	0.897 ± 0.084	0.881 ± 0.007
− loss gene	0.975 ± 0.006	4.677 ± 4.009	0.908 ± 0.055	0.882 ± 0.007
− f_3	0.962 ± 0.030	1.618 ± 0.950	0.883 ± 0.059	0.872 ± 0.025
f_3 with MSP	0.969 ± 0.014	3.287 ± 2.081	0.912 ± 0.029	0.878 ± 0.014
easy pseudo-unknown (Recon)	0.977 ± 0.003	0.737 ± 0.515	0.889 ± 0.048	0.887 ± 0.006

candidates rises from about 45–60% in the first generation to about 85% by generation 5 for both variants. Most of the drop in the best error happens within the first ten generations (Figure 1(b)). The SC-MODEO-CNN curve lies higher only because macro F1 is a harder target than weighted F1; it does not indicate worse models, and the two methods are compared on equal terms in Table 3.

Pooled over five seeds, the final MODEO-CNN archives contain 29 members and the SC-MODEO-CNN archives 74 (Figure 2). The third objective explains the difference: it keeps models that separate the pseudo-unknown well even when they are neither the smallest nor the most accurate.

6.2 Closed-set classification

SC-MODEO-CNN reaches a macro-F1 of 0.977 ± 0.004 against 0.974 ± 0.005 for MODEO-CNN (Table 3). With $p = 0.125$, this is a trend rather than a significant improvement. Model size is more consistent across seeds: 1.34 ± 0.21 MFLOPs against 1.07 ± 1.00. The hand-designed CNN (0.980) and the LSTM (0.978) perform on par with both searched models. A likely reason is that candidates are ranked after only five epochs on 30% of the data, which makes the fitness estimates noisy. The search nevertheless offers an automatic design process and a set of models covering the trade-off between size and accuracy, rather than a single fixed network.

6.3 Zero-day detection of DoS

Scores computed from the classifier output alone fail to detect DoS: MSP and energy reach AUROC values of only 0.54–0.61, close to chance (Figure 3). The Mahalanobis score on the embedding is much better, with AUROC 0.928 for MODEO-CNN and 0.897 for SC-MODEO-CNN, yet it still accepts 21–42% of DoS records as known traffic (Table 4). Adding the autoencoder and fusing the scores raises AUROC to 0.98 and lowers FPR@95 to about 0.04, so roughly ten times fewer DoS records are missed. However, the fused score performs about the same as the autoencoder alone, which means the gain comes from the autoencoder and the embedding adds little for DoS.

Thresholding matters as much as the score itself. With a single global threshold, rare known classes are rejected too often: MSCI reaches an F1 of only 0.46, MFCI 0.00 and NMRI 0.68, and open-set macro-F1 stays at 0.66–0.69 (Figure 4). With per-class thresholds, these rise to 0.93, 0.94 and 0.93, open-set macro-F1 reaches 0.88–0.89 and 97% of DoS records are detected. The F1 for DoS itself is only 0.42, because DoS makes up just 1.9% of the traffic: the 5% false-alarm rate on known traffic produces more false alerts than there are true DoS alarms. In practice, an operator would choose a stricter operating point.

Table 6: Faithfulness and stability of explanations for the 10 SC-MODEO-CNN models (5 seeds $\times$ 2 Pareto picks).

Method	Deletion AUC ↓	(random)	Insertion AUC ↑	(random)	Kendall τ	Top-5 overlap
Integrated Gradients	0.537 ± 0.118	0.663 ± 0.064	0.820 ± 0.073	0.665 ± 0.055	0.72 ± 0.08	0.63
GradientSHAP	0.543 ± 0.112	0.663 ± 0.064	0.814 ± 0.067	0.665 ± 0.055	0.68 ± 0.10	0.64

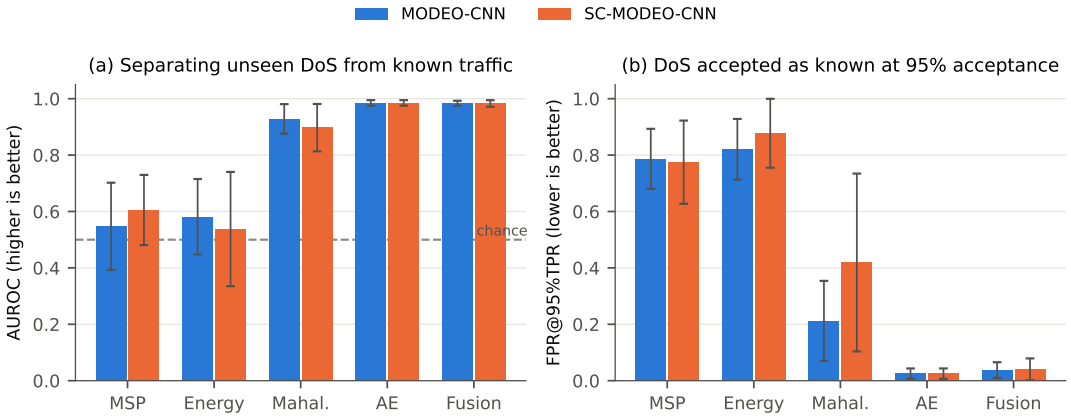


Figure 3: Detecting the unseen DoS attack with five post-hoc scores (mean $\pm$ s.d., 5 seeds).

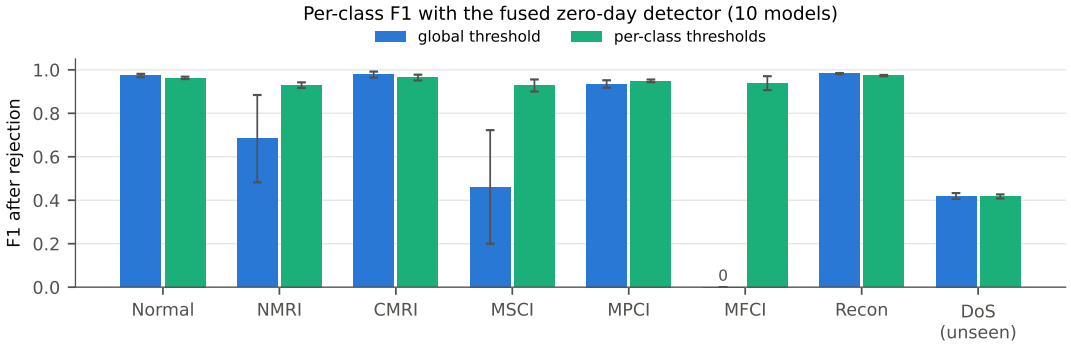


Figure 4: Per-class F1 after rejection with the fused detector: global vs. per-class thresholds (10 models).

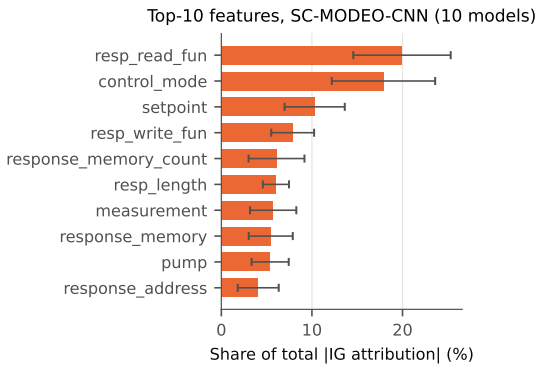


Figure 5: Top-10 features by Integrated Gradients attribution (10 SC-MODEO-CNN models).

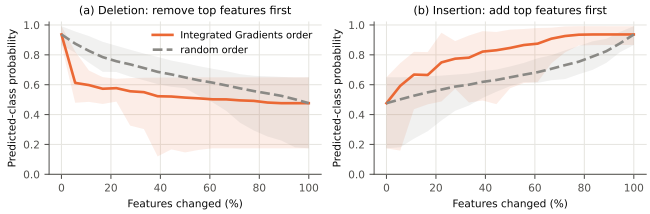


Figure 6: Deletion and insertion curves: attribution order vs. random order.

6.4 Ablation study

Table 5 isolates each component. Removing f_3 lowers both macro-F1 (0.962) and Mahalanobis AUROC (0.883). Removing the loss gene leaves F1 nearly unchanged but produces models about 3.5 times larger (4.68 against 1.34 MFLOPs). With the easy pseudo-unknown (Recon), f_3 is always zero, so the third objective provides no signal at all. Two results call for

caution. In 16 of 20 runs the search chose an imbalance-aware loss, which matches the 80% expected by chance (four of the five loss options are imbalance-aware), so we do not claim a preference. Also, f_3 learned on CMRI did not beat MODEO-CNN on DoS (Mahalanobis AUROC 0.897 against 0.928), indicating that separability learned on a command-injection attack does not transfer to DoS.

6.5 Explainability

The attributions are faithful to the models (Table 6, Figure 6). Removing the top-attributed features first lowers the predicted-class probability faster than random removal (deletion AUC 0.54 against 0.66), and adding them first raises it faster (insertion AUC 0.82 against 0.67). Rankings are also fairly stable across models, with Kendall τ of 0.72 for IG and 0.68 for GradientSHAP, and a top-5 overlap of 0.63–0.64. The most important features are `resp_read_fun` (20% of total attribution), `control_mode` (18%) and `setpoint` (10%) (Figure 5). These match the attack mechanics: function-code fields relate to function-code and command injection, setpoint and control mode to state and parameter injection, and the measurement field to response injection.

7 Discussion and Limitations

Three choices worked well. Separating detection, handled by the autoencoder and the embedding distance, from classification made zero-day detection far more reliable than relying on classifier confidence. Per-class calibration of thresholds was what turned a good score into usable open-set detection, because it stopped rare known attacks from being rejected. The changes to the search itself mainly improved accuracy and model size.

Other parts did not work as hoped. The zero-day objective, learned on a single attack type, did not carry over to DoS, and the closed-set gain was small. Two directions may help: using several pseudo-unknown classes during search, or replacing the AUROC-based f_3 with a distance-based objective that directly shapes the embedding.

The study has several limitations. Only one unseen attack (DoS) was evaluated. With five seeds, no comparison can reach significance at $p < 0.05$. The search budget was reduced relative to the original paper. Results come from a single dataset, and should be confirmed on others such as Edge-IIoTset [19] and BoT-IoT [20]. Finally, for rare unknown attacks, any fixed operating point trades detections against false alarms, and that choice must be made by the operator.

8 Conclusion

Intrusion detectors produced by architecture search for IIoT, such as MODEO-CNN, handle class imbalance poorly, cannot recognise attacks absent from training, and give no explanation for their alerts. SC-MODEO-CNN addresses these gaps with an imbalance-aware search that includes a zero-day objective, a fused detector with per-class thresholds and verified attribution-based explanations. On Gas Pipeline it reaches a macro-F1 of 0.977, misses about ten times fewer unseen DoS records and raises open-set macro-F1 to 0.88–0.89. Future work will apply a leave-one-attack-out protocol to all attack classes, use more seeds, add a second dataset and run the search at the full budget.

References

[1] K.-D. Lu, J.-C. Huang, G.-Q. Zeng, M.-R. Chen, G.-G. Geng, and J. Weng, "Multi-objective discrete extremal optimization of variable-length blocks-based CNN by joint NAS and HPO for intrusion detection in IIoT," *IEEE Transactions on Dependable and Secure Computing*, vol. 22, no. 4, pp. 4266–4283, 2025.

[2] J.-C. Huang, G.-Q. Zeng, G.-G. Geng, J. Weng, and K.-D. Lu, "SOPA-GA-CNN: Synchronous optimisation of parameters and architectures by genetic algorithms with convolutional neural network blocks for securing Industrial Internet-of-Things," *IET Cyber-Systems and Robotics*, vol. 5, no. 1, p. e12085, 2023.

[3] W. J. Scheirer, A. de Rezende Rocha, A. Sapkota, and T. E. Boult, "Toward open set recognition," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 35, no. 7, pp. 1757–1772, 2013.

[4] Y. Sun, B. Xue, M. Zhang, G. G. Yen, and J. Lv, "Automatically designing CNN architectures using the genetic algorithm for image classification," *IEEE Transactions on Cybernetics*, vol. 50, no. 9, pp. 3840–3854, 2020.

[5] S. Boettcher and A. Percus, "Nature's way of optimizing," *Artificial Intelligence*, vol. 119, no. 1–2, pp. 275–286, 2000.

[6] K. Deb, A. Pratap, S. Agarwal, and T. Meyarivan, "A fast and elitist multiobjective genetic algorithm: NSGA-II," *IEEE Transactions on Evolutionary Computation*, vol. 6, no. 2, pp. 182–197, 2002.

[7] D. Hendrycks and K. Gimpel, "A baseline for detecting misclassified and out-of-distribution examples in neural networks," in *International Conference on Learning Representations (ICLR)*, 2017.

[8] W. Liu, X. Wang, J. D. Owens, and Y. Li, "Energy-based out-of-distribution detection," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020.

[9] K. Lee, K. Lee, H. Lee, and J. Shin, "A simple unified framework for detecting out-of-distribution samples and adversarial attacks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 31, 2018.

[10] Y. Mirsky, T. Doitshman, Y. Elovici, and A. Shabtai, "Kitsune: An ensemble of autoencoders for online network intrusion detection," in *Network and Distributed System Security Symposium (NDSS)*, 2018.

[11] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal loss for dense object detection," in *IEEE International Conference on Computer Vision (ICCV)*, pp. 2980–2988, 2017.

[12] Y. Cui, M. Jia, T.-Y. Lin, Y. Song, and S. Belongie, "Class-balanced loss based on effective number of samples," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 9268–9277, 2019.

[13] M. Sundararajan, A. Taly, and Q. Yan, "Axiomatic attribution for deep networks," in *International Conference on Machine Learning (ICML)*, pp. 3319–3328, 2017.

[14] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.

[15] V. Petsiuk, A. Das, and K. Saenko, "RISE: Randomized input sampling for explanation of black-box models," in *British Machine Vision Conference (BMVC)*, 2018.

[16] N. Kokhlikyan, V. Miglani, M. Martin, E. Wang, B. Alsallakh, J. Reynolds, A. Melnikov, N. Kliushkina, C. Araya, S. Yan, and O. Reblitz-Richardson, "Captum: A unified and generic model interpretability library for PyTorch." arXiv:2009.07896, 2020.

[17] T. Morris and W. Gao, "Industrial control system traffic data sets for intrusion detection research," in *Critical Infrastructure Protection VIII (ICCIP 2014)*, vol. 441 of *IFIP Advances in Information and Communication Technology*, pp. 65–78, Springer, 2014.

[18] F. Wilcoxon, "Individual comparisons by ranking methods," *Biometrics Bulletin*, vol. 1, no. 6, pp. 80–83, 1945.

[19] M. A. Ferrag, O. Friha, D. Hamouda, L. Maglaras, and H. Janicke, "Edge-IIoTset: A new comprehensive realistic cyber security dataset of IoT and IIoT applications for centralized and federated learning," *IEEE Access*, vol. 10, pp. 40281–40306, 2022.

[20] N. Koroniotis, N. Moustafa, E. Sitnikova, and B. Turnbull, "Towards the development of realistic botnet dataset in the Internet of Things for network forensic analytics: Bot-IoT dataset," *Future Generation Computer Systems*, vol. 100, pp. 779–796, 2019.